

Pokédex App v0.1.0

Este proyecto es una Pokédex local construida con Streamlit que usa un backend JSON local (**json-server**) y datos de la PokéAPI.

Objetivo de esta actualización

- No necesitas **Conda**.
- Funciona con **Python base** (3.8+ recomendado).
- Incluye manual de arquitectura de la aplicación (clases, funciones y flujo).

Requisitos (Python base)

- Python 3.8+ instalado en el sistema.
- Node.js + npm (solo para json-server). Opcional: si no quieres json-server, usa fallback directo a PokéAPI.

Instalación rápida (Windows)

1. Abre PowerShell o CMD.
2. Ve a la carpeta del proyecto:

```
cd "c:\Users\ingga\OneDrive\Documentos\Nueva carpeta\Administración de  
Sistemas de Información\data analisis\Pokedex\versión 0_0_1"
```

3. (Opcional pero recomendado) Crear entorno virtual Python:

```
python -m venv .venv  
.venv\Scripts\activate
```

4. Instalar dependencias Python:

```
python -m pip install --upgrade pip  
python -m pip install -r requirements.txt
```

5. Instalar json-server globalmente (requiere Node.js):

```
npm install -g json-server
```

Cómo ejecutar

Opción A (recomendado, local)

1. Abrir terminal A y ejecutar:

```
start_server.bat
```

2. Abrir terminal B y ejecutar:

```
run_app.bat
```

Opción B (manual)

1. Iniciar json-server:

```
json-server --watch data/db.json --port 3000
```

2. Iniciar Streamlit:

```
streamlit run app.py
```

Archivos clave y diseño de la aplicación

1) app.py (entrada principal)

- Configura la UI de Streamlit y gestiona el layout.
- Usa `initialize_session_state()` para sesión, `render_search_section()` para búsqueda y `render_results_section()`.
- Llama a controladores y vistas.

2) Modelo (data):

- `models/pokemon_model.py`: funciones para leer datos JSON y buscar Pokémon.
- `models/favorites_model.py`: carga y guarda favoritos en `data/favorites.json`.
- `models/history_model.py`: guarda historial en `data/history.json` y obtiene populares.

3) Controladores (lógica):

- `controllers/pokemon_controller.py`: funciones `handle_search()`, `handle_random_pokemon()`, `handle_favorite_toggle()`.
- `controllers/chart_controller.py`: genera gráficos tipo radar desde stats.

4) Vistas (UI):

- `views/layout.py`: configura Streamlit, encabezado, pie de página y carga CSS.
- `views/pokemon_view.py`: renderiza tarjeta de Pokémon, lista de favoritos e historial.

5) Utilidades:

- `utils/constants.py`: constantes globales (`POPULAR_POKEMON`, rutas, etc.).
- `utils/helpers.py`: funciones de utilidades (formateo, conversiones, etc.).

6) Datos JSON:

- `data/db.json`: datos de pokemon (obtenidos por `scripts/populate_db.py`).
- `data/favorites.json`: favoritos de usuario.
- `data/history.json`: historial local.

7) Script de datos:

- `scripts/populate_db.py`: pobla `data/db.json` consultando PokéAPI por generación.

Flujo de ejecución principal

1. El usuario abre la app y escribe un nombre/ID.
2. Se llama `handle_search()` en `controllers/pokemon_controller.py`.
3. Este obtiene datos desde `models/pokemon_model.py` (json-server local / fallback PokéAPI).
4. La vista muestra la tarjeta del Pokémon y botones de favorito.
5. Favoritos e historial se guardan en archivos JSON.

Cambios clave aplicados para usar Python base (sin Conda)

- `run_app.bat`: ya no exige `conda activate`. Si existe `.venv`, lo activa; si no, usa el Python global.
- README actualizado para instrucción con Python base.
- Se reorganizó el manual para incluir descripción de módulos/clases/funciones.

Nota importante

- Si no tienes `json-server`, la app funciona en modo fallback consultando la PokéAPI directamente, pero la búsqueda local es más rápida con `json-server`.
- Asegúrate de tener puertos libres y de ejecutar `populate_db.py` si tu `data/db.json` está vacío.

Comandos de utilidad

- Poblar datos (generación 1):

```
python scripts/populate_db.py --generation 1
```

- Reiniciar datos: elimina `data/db.json` y vuelve a poblar.

Estado actual

- Aplicación funcional con Python base y Streamlit.
- Entorno conda ya no es obligatorio.

Este README es el manual principal del proyecto.